

Day-64-Git-and-GitHub-Fundamentals

1. Day Number

Day 64

2. Topic Name

Git, Repository, Commit, and GitHub

3. Connection

You now have a small Python project with files such as `main.py` and `requirements.txt`.

Today, you will learn how developers **track changes** to those project files using **Git**.

The basic workflow is:

```
Python Project
  ↓
Initialize Git
  ↓
Check Changes
  ↓
Select Files
  ↓
Commit Changes
  ↓
Later → Upload/Push to GitHub
```

4. Important Topics

The main ideas for today are:

- **Git repository** — a project folder whose changes Git tracks
- `git init` — starts Git in a folder
- `git status` — shows the current state of files
- `git add` — selects changes for the next commit
- `git commit` — saves a snapshot of selected changes
- **GitHub repository** — an online location for a Git project
- **push** — sending local Git commits to a remote repository such as GitHub

5. Foundational Notes

What is Git?

Git is a version-control system.

Suppose your Python program originally contains:

```
print("Hello")
```

Later you change it to:

```
print("Hello, Python!")
```

Git can help you keep a history of changes like this.

Think of Git as a collection of **checkpoints for your project**.

Instead of creating folders such as:

```
project_final  
project_final2  
project_really_final  
project_final_fixed
```

Git lets you keep one project while recording meaningful versions of it.

What is a repository?

A **repository**, often shortened to **repo**, is a project that Git is tracking.

For example:

```
my_project/  
|  
├─ main.py  
├─ requirements.txt  
└─ data/
```

After running:

```
git init
```

inside `my_project`, the folder becomes a Git repository.

Git creates a hidden folder named:

```
.git
```

That folder stores Git's tracking information.

6. Git vs GitHub

Git and GitHub are related, but they are **not the same thing**.

Git	GitHub
Version-control tool	Website/service for hosting Git repositories
Works on your computer	Primarily stores repositories online
Tracks changes	Helps share and collaborate on projects
Creates commits	Stores commits that you push
Can work without internet	Usually requires internet for syncing

A simple way to remember it:

```
Git = tracks your project
```

```
GitHub = stores/shares your Git project online
```

You can use **Git without GitHub**.

For today's exercise, concentrate on **local Git first**.

7. Easy Example

Imagine this project:

```
hello_project/
|
└─ main.py
```

main.py contains:

```
print("Hello, Git!")
```

Open a terminal inside hello_project.

First initialize Git:

```
git init
```

Then check the project:

```
git status
```

Git may report that main.py is currently **untracked**.

That means:

| Git sees the file, but you have not yet selected it for version tracking.

Select it:

```
git add main.py
```

Check again:

```
git status
```

Now `main.py` should be ready to commit.

Create a commit:

```
git commit -m "Add main Python file"
```

The text:

```
Add main Python file
```

is the **commit message**.

It describes what this checkpoint contains.

8. Problem Statement

You have a small Python project.

Your task is to:

1. Open the project folder in a terminal.
2. Initialize Git in the project.
3. Check Git's status.
4. Add `main.py`.
5. Check the status again.
6. Create your **first commit**.
7. Understand conceptually how the repository could later be pushed to GitHub.

Keep everything local for now.

9. Concepts Used

For this exercise you will use:

```
Repository
```

```
↓
```

```
git init
```

```
Project state
```

```
↓
git status

Select a file
↓
git add

Save checkpoint
↓
git commit
```

The basic Git workflow is:

```
Modify files
↓
git status
↓
git add
↓
git commit
```

You will repeat this pattern frequently while developing projects.

10. Thought Process

Suppose you have:

```
my_python_project/
|
├── main.py
├── requirements.txt
└── data/
```

First ask:

Is Git already tracking this project?

If not:

```
Initialize repository
```

Next ask:

What files have changed or are untracked?

Use:

```
git status
```

Next ask:

Which changes do I want in my next checkpoint?

Select them with:

```
git add
```

Finally ask:

What meaningful description should this checkpoint have?

Create a commit:

```
git commit -m "..."
```

This creates a saved point in your project's Git history.

11. Beginner-Friendly Command Steps

Step 1 — Enter the project folder

Conceptually:

```
cd my_python_project
```

Your terminal should be working inside the project folder.

Step 2 — Start Git

```
git init
```

This turns the folder into a Git repository.

Step 3 — Check status

```
git status
```

Look for `main.py`.

Initially it may appear as an **untracked file**.

Step 4 — Select `main.py`

```
git add main.py
```

This tells Git:

| Include the current version of `main.py` in my next commit.

Step 5 — Check again

```
git status
```

Notice how the status of `main.py` has changed.

Step 6 — Create the first commit

The command structure is:

```
git commit -m "your message"
```

Use a short message explaining what you saved.

For example, a first commit could describe adding the initial Python program.

What Does `git add` Really Do?

A common beginner misunderstanding is:

```
git add = save permanently
```

Not quite.

Think of three stages:

```
Working files
  ↓
  git add
  ↓
Staging area
  ↓
  git commit
  ↓
Git history
```

So:

```
git add main.py
```

means:

| Prepare this version of `main.py` for the next commit.

And:

```
git commit
```

means:

| Record the prepared changes as a checkpoint.

What Is a Commit?

A **commit** is a recorded snapshot/checkpoint in your project's history.

For example:

```
Commit 1
Initial Python program

      ↓

Commit 2
Add input validation

      ↓

Commit 3
Fix calculation bug
```

Each commit lets you build a history of how the project developed.

Commit messages should briefly explain the change.

Good beginner examples:

```
Add initial Python program
```

```
Add requirements file
```

```
Fix input handling
```

Less useful:

```
stuff
```

or:

```
changes
```

12. Suggested Solving Approach — Local Git First

For now, remember this simple workflow:

```
git init
git status
git add <file>
git status
git commit -m "message"
```

Do not worry about GitHub yet.

First become comfortable with:

```
See changes
  ↓
Choose changes
  ↓
Commit changes
```

Once local Git makes sense, GitHub becomes much easier to understand.

How GitHub Fits In Later

Imagine your computer contains this Git repository:

```
Your Computer

my_python_project
  |
  | commits
  ▼
Local Git Repository
```

Later, you can create an empty repository on **GitHub** and connect your local repository to it.

Conceptually:

Your Computer		GitHub
Local Git Repository	—push—▶	Online Repository

A **push** sends your local commits to the connected remote repository.

For today, you only need to understand:

```
commit = save checkpoint locally

push = send commits to GitHub
```

You do **not** need to push anything for today's exercise.

13. Easy Edge Cases

File is not tracked

You run:

```
git status
```

and see `main.py` listed as untracked.

That is normal for a new repository.

Git has discovered the file but it has not yet been selected.

Think:

```
Untracked
  ↓
git add
  ↓
Staged
```

Nothing to commit

Sometimes you may run a commit command and Git reports something similar to:

```
nothing to commit
```

This usually means Git cannot find any new staged changes.

Possible reasons include:

```
You already committed the changes.
```

or:

```
You changed nothing since the previous commit.
```

or:

```
You changed a file but did not git add it.
```

Use:

```
git status
```

to understand the situation.

14. Expected Result

By the end of the exercise, your project should still look something like:

```
my_python_project/  
|  
├── main.py  
├── requirements.txt  
└── data/
```

But internally, it is now also a **Git repository**.

Conceptually:

```
my_python_project/  
|  
├── .git/           ← Git's internal information  
├── main.py  
├── requirements.txt  
└── data/
```

And Git should contain at least one commit representing your first project checkpoint.

Your workflow should now make sense as:

```
Project  
  ↓  
git init  
  ↓  
Repository  
  ↓  
git status  
  ↓  
See changes  
  ↓  
git add  
  ↓  
Stage changes  
  ↓  
git commit  
  ↓  
Saved checkpoint
```

Later:

```
Local commits  
  ↓  
push
```

↓
GitHub repository

15. Hint Only

For today's exercise, remember these four ideas:

Start tracking the project

↓

`git init`

See what Git sees

↓

`git status`

Prepare `main.py`

↓

`git add ...`

Create a checkpoint

↓

`git commit ...`

If you get confused at any point, run:

`git status`

It is one of the most useful Git commands for beginners.